# altairsim — CDOS 2.58 — Cromemco's disk operating system, off a 16FDC

2026-07-31 · ecff624

**CDOS** was Cromemco's operating system for its Z80 S-100 machines: a close work-alike of Digital Research's CP/M (it runs many `.COM` programs unchanged) with Cromemco's own extensions. This example boots CDOS 2.58 from an 8″ diskette through a **Cromemco 16FDC** floppy controller — the card that carried the disk controller, the console UART, and the boot PROM all on one board.

```
cd examples/cdos
altairsim cdos.toml
```

The machine is a full 4 MHz Cromemco: a Z80, 64K of RAM, and one 16FDC with the CDOS master in drive A. There is nothing else to configure — `bootstrap = true` arms the boot PROM and the machine comes up loading CDOS.

## Booting it

```
Preparing to boot, ESC to abort
Standby
CDOS version 02.58
Cromemco Disk Operating System
Copyright (C) 1977, 1983 Cromemco, Inc.

A.
```

CDOS loads and comes straight up to its `A.` prompt — **no keypress needed**. This machine's 16FDC is strapped for its fixed 300-baud *modem* console (the board's switch 5): that setting tells RDOS to skip the terminal auto-baud, which on real hardware wanted a RETURN to measure your terminal's speed. Here the prompt just appears.

The `A.` prompt is CDOS waiting on drive A, the same idea as CP/M's `A>`. Try:

```
DIR       list the files on the disk
STAT      system status: memory, devices, the disk label and its date
```

`DIR` lists the 18 files on the master — the assembler (`ASMB`, `LINK`), the editor (`EDIT`), system utilities (`XFER`, `INIT`, `DUMP`, `STAT`), `XMODEM`, and CDOS itself. `STAT` reads the disk's own label

sector back and prints `CDOS2.58`, dated `05-19-82`.

## How the boot works

Reset arms the **RDOS 2.52** boot PROM in the 16FDC's window at C000; the machine's `startup` list runs it. RDOS reads the boot track through the WD FD1793 controller, loads the CDOS cold loader, and the loader pulls `CDOS.COM` into memory and jumps into it. CDOS then relocates itself into the top of RAM and switches the PROM out (`OUT 40H`), which is why the memory board shadows the PROM window on **reads only** — so the boot sees the PROM, but CDOS can put running code in the RAM underneath.

## Two things that are authentic, not settings

- **4 MHz.** A real Cromemco ran at 4 MHz, and this disk needs it: RDOS's double-density read loop only just keeps up with a 500 kbit/s diskette at 4 MHz. At 2 MHz the controller reports Lost Data and the boot fails. The `clock_hz = 4000000` in the `.toml` is the real machine, not a tuning knob.
- **The mixed-density diskette.** `CDOS258-8IN-DSDD.DSK` is an 8″ **double-sided** image whose track 0 is single density (so the boot PROM, which only knows single density, can read it) and whose remaining tracks are double density. The 16FDC recognises the format from the image — you do not tell it.

## Console speed

By default the console runs at **full speed** — output appears as fast as CDOS prints it, regardless of the 300-baud modem strap. If you want the authentic feel of a 300-baud modem console (deliberate, and slow — a `DIR` takes many seconds), set the rate on the tty:

```
[board.unit.tty]
connect = "console"
rate    = "real"    # pace the line in wall-clock time at the programmed baud
```

`rate = "full"` (the default) never paces the emulated line; `rate = "real"` does. Either way a real serial *port* on the other end still clocks its own wire — `rate` governs only the emulated console's internal timing.

## A word about the disk

`CDOS258-8IN-DSDD.DSK` is **tracked in the repository** and mounted read/write in drive A, which is what a real machine is. Booting, `DIR` and `STAT` only read it. If you are about to test writes in anger, copy the image first, or add `writeprotect = true` to the drive in the `.toml`. In a clone, `git checkout` restores a dirtied image; in a release package there is no such safety net.